



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

www.rvce.edu.in
Tel: +91-80-68188100
+91-80-68188111
+91-80-68188112

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Automated Fruit Freshness Detection for Inventory Management

Machine Learning Operations – Tutorials

REPORT

Submitted by

Alroy Deon Saldanha **1RV23AI012**

Aniket R T **1RV23AI017**

Anjali Suresh Kalarikkal **1RV23AI018**

Apoorva Krishna P **1RV23AI020**

Introduction	4
Problem Definition	5
Business Objectives	5
Overview of MLOps tools	5
Exploratory Data Analysis	8
Dataset Overview	8
Image Metadata Analysis	8
Distribution of Image Widths	8
Distribution of Image Heights	8
Aspect Ratio Distribution	8
Class Distribution Analysis	9
EDA Insights and Implications	10
Models	11
Model Architecture Details	11
1. Convolutional and Pooling Layers	11
2. Fully Connected Layers	11
3. Model Size	11
Implementation Details	12
Input Processing	12
Data Augmentation	12
Loss Function	12
Optimizer	12
Training Configuration	12
Training Performance and Evaluation	12
Results after 3 epochs	12
Epoch 3	12
Performance Interpretation	13
Why This Model Is a Best Fit for the Problem	13
Experiment Tracking	14
How MLflow Is Used	14
1. Experiment Management	14
2. Parameter Tracking	14
3. Metric Logging	14
4. Model Versioning	14
5. Deployment Support	15
Significance in an MLOps Pipeline	15
Data versioning	16
System Architecture and Technology Stack	16
Dataset Tracking and Structure	16
Pipeline Architecture and Reproducibility	16
Data Validation and Analysis	16

Version Control Workflow	17
Remote Storage and Collaboration	17
Documentation and Usability	17
Advantages of the DVC-Based Approach	17
Containerization Using Docker	19
Production Monitoring and Infrastructure Observability	20
1. Traffic Patterns and Request Latency	20
2. Latency Breakdown and Cold Start Analysis	20
3. Resource Utilization (CPU & Memory)	21
4. Autoscaling and Instance Management	22
5. Network Throughput	22
Significance	23
CI/CD	24
Logging and Dashboard	25
Overview	25
Technology Stack	25
Logging Workflow	25
Dashboard Visualization	26
Significance	26
Persistent Data Storage (Hugging Face Datasets)	28
Baseline Behavior & Feature Drift Detection in CNN for Fruit Freshness	29
1. Baseline Prediction Behavior	29
Baseline Prediction Distribution	29
2. Baseline Confidence Statistics	29
3. Feature Drift Detection Setup	29
Drift Detection Methodology	29
Role of Alibi Detect in Drift Detection	30
How Alibi Detect Works (KSDrift)	30
How It Was Applied in This Project	30
Feature Drift Detection Results	30
Overall Drift Outcome	30
Interpretation of Results	30
Visual Evidence of Drift	31
Key Takeaway	31
Model Explainability Using Grad-CAM	32
1. Purpose of Explainability	32
2. Explainability Method: Grad-CAM	32
3. Experimental Setup	32
5. Interpretation of the Output	32
6. Relationship to Drift Detection Results	33
7. Key Insight	33
Conclusion	33



Introduction

In modern agricultural supply chains and warehouse operations, maintaining the freshness and quality of fruits is critical for minimizing waste, ensuring food safety, and maximizing economic value. Manual inspection of fruits is time-consuming, inconsistent, and prone to human error, especially in high-throughput warehouse environments where large volumes of produce move rapidly on conveyor belts. With increasing demand for automation and real-time quality assessment, computer vision and deep learning provide an effective solution for intelligent fruit inspection systems.

This project presents an automated fruit freshness detection system that leverages Convolutional Neural Networks (CNNs) to classify fruits as *fresh* or *rotten* while they move on a conveyor belt. Upon detection, rotten fruits are automatically removed from the conveyor belt, enabling efficient segregation and real-time quality control. The system is integrated with a warehouse inventory management system, allowing dynamic updates to stock quality and quantity.

The motivation for this project arises from the need to reduce post-harvest losses, improve warehouse operational efficiency, and support scalable AI deployment in real-world environments. A significant portion of fruits is wasted due to delayed or inaccurate quality assessment, leading to economic losses and sustainability concerns. Traditional machine learning models often fail when deployed due to data drift, lack of monitoring, and absence of explainability.

To address these challenges, this project goes beyond model development and adopts a complete MLOps lifecycle, ensuring reproducibility, scalability, transparency, and reliability of the deployed system. By incorporating tools for experiment tracking, version control, explainable AI, CI/CD automation, monitoring, bias evaluation, and documentation, the system is designed to be production-ready for warehouse-scale deployment rather than remaining a proof-of-concept.

Problem Definition

Warehouses handling fresh produce require a real-time, automated mechanism to detect and remove rotten fruits from conveyor belts while maintaining accurate inventory records. Existing manual or semi-automated systems suffer from inefficiencies, inconsistent quality checks, lack of scalability, and absence of model governance.

The core problem addressed in this project is the design and deployment of a robust, end-to-end MLOps-based computer vision system that:

- Accurately classifies fruits as fresh or rotten in real time,
- Operates reliably under changing data conditions,
- Provides explainability for model decisions,
- Monitors model performance and data drift,
- Ensures fairness, traceability, and reproducibility,
- Integrates seamlessly with warehouse inventory systems.

Business Objectives

- Develop a CNN-based fruit freshness detection model.
- Implement automated fruit removal by triggering mechanical or control actions when rotten fruits are detected.
- Feature drift detection and transparency through explainability
- Adherence to MLOps lifecycle for development and deployment.

Overview of MLOps tools

This project implements an end-to-end MLOps pipeline integrating data versioning, experiment tracking, explainability, drift detection, deployment, and monitoring to ensure reliable and adaptive CNN-based fruit freshness classification.

Component	Tools Used	Purpose	Application
Data Management	DVC (Data Version Control)	Versions and manages large datasets separately from code	• Tracks multiple versions of fruit image datasets (training, validation, updated data) • Links each trained model to the exact dataset used • Enables comparison of model behavior before and after drift-based retraining

Experiment Tracking & Version Control	MLflow	Logs experiments, model parameters, metrics, and artifacts	• Stores training configurations (epochs, learning rate, architecture) • Logs metrics before and after retraining triggered by drift • Maintains multiple model versions for baseline vs updated comparison
Model Explainability	Grad-CAM	Visualizes important image regions influencing CNN predictions	• Helps interpret CNN decisions by highlighting attention regions • Validates drift impact by showing attention shift under degraded behavior • Used to analyze model focus before retraining
Model Deployment & Serving	Google Cloud (Backend) Vercel (Frontend)	Deploys and serves the model for real-time inference	• Backend handles model inference and drift detection logic • Frontend allows users to upload fruit images and view predictions • Enables real-time model usage and monitoring
Monitoring, Governance & Collaboration	Hugging Face Spaces GitHub Kaggle	Supports monitoring, logging, and team collaboration	• Hugging Face Spaces hosts live demos and logs prediction outputs • Enables monitoring of confidence scores and drift signals • GitHub manages code versioning and collaboration • Kaggle supports dataset access and experimentation
Drift Detection	Alibi Detect	Statistically detects data and prediction drift	• Detects feature drift using CNN embeddings • Detects prediction and confidence drift using KS tests • Triggers retraining when significant drift is detected

			Ensures model adapts to changing data distributions
Containerization	Docker	Packages application with all dependencies	• Ensures consistent environments across training and deployment • Simplifies deployment of model and monitoring pipeline • Improves reproducibility and scalability



Exploratory Data Analysis

Exploratory Data Analysis was conducted to understand the structure, quality, and distribution of the fruit image dataset before model development. The analysis focused on metadata inspection, image dimension characteristics, aspect ratio variation, and class-wise distribution to inform preprocessing and modeling decisions.

Dataset Overview

The dataset consists of 14 classes, representing *fresh* and *rotten* conditions across 7 fruit categories (apples, bananas, cucumbers, okra, oranges, potatoes, and tomatoes). Each image is stored in RGB format and is associated with metadata such as filename, width, height, and class label.

Total number of classes: 14

Image mode: RGB

This multi-class structure supports both fine-grained classification (fruit + condition) and binary freshness detection (fresh vs. rotten) use cases.

Image Metadata Analysis

A preview of the metadata indicates variability in image dimensions, with widths and heights differing across samples. This reflects real-world image acquisition conditions rather than artificially standardized inputs, making the dataset suitable for deployment-focused deep learning models.

Distribution of Image Widths

The image width distribution shows:

- A wide range of widths, approximately from 150 pixels to over 800 pixels.
- Prominent peaks around 400–500 pixels, indicating common capture resolutions.
- A right-skewed distribution, suggesting the presence of higher-resolution images in smaller quantities.

This variability highlights the necessity of image resizing and normalization during preprocessing to ensure consistent CNN input dimensions.

Distribution of Image Heights

The height distribution exhibits:

- Heights ranging roughly from 120 pixels to 500+ pixels.
- Strong concentration between 300–450 pixels, forming the dominant resolution range.
- Minor secondary peaks corresponding to alternate capture settings.

The variation in height, similar to width, reinforces the need for uniform resizing strategies such as center cropping or padding while preserving important visual features.

Aspect Ratio Distribution

Aspect ratio analysis reveals:

- Most images fall within an aspect ratio range of 0.8 to 1.5, with clear peaks near 1.0 (square images) and ~1.4 (rectangular images).
- A long tail extending beyond 2.0, representing elongated images.

This distribution suggests that non-uniform aspect ratios are common, and careless resizing could distort fruit features. Therefore, aspect-ratio-preserving resizing combined with padding is recommended to maintain spatial integrity.

Class Distribution Analysis

The dataset shows class imbalance across fruit types and freshness conditions:

Class	Number of Images
freshapples	2424
rottenapples	3248
freshbanana	2468
rottenbanana	2932
freshtomato	1858
rottentomato	1825
freshoranges	1466
rottenoranges	1595
freshpotato	536
rottenpotato	802
freshokra	635
rottenokra	338
freshcucumber	496
rottencucumber	421



Key observations:

- Apple and banana classes dominate the dataset.
- Okra and cucumber classes are comparatively underrepresented.
- Rotten samples are slightly more prevalent than fresh samples in several fruit categories.

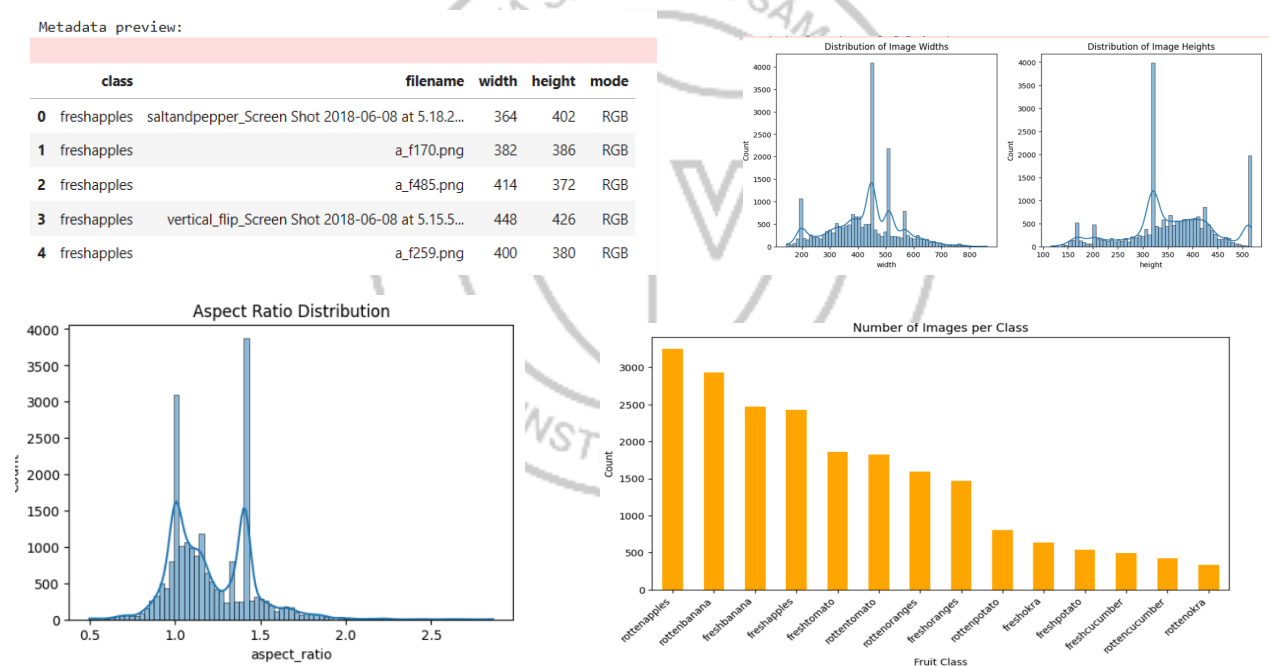
This imbalance can bias the model toward majority classes if not addressed. As a result, class weighting, data augmentation, and fairness evaluation (Fairlearn) are essential components of the training pipeline.

EDA Insights and Implications

From the EDA, the following conclusions were drawn:

1. Image dimensions vary significantly, requiring standardized preprocessing.
2. Aspect ratios are inconsistent, necessitating careful resizing to avoid feature distortion.
3. Class imbalance exists, motivating the use of bias mitigation techniques.
4. The dataset realistically reflects warehouse and conveyor belt conditions, strengthening the model's real-world applicability.
5. Most images are in bright colors implying that the images are from daytime and results may vary in poor lighting.

These findings directly informed preprocessing choices, CNN architecture design, fairness evaluation, and monitoring strategies in the MLOps pipeline.



Corrupted or unreadable images: 0

Computing RGB stats: 100% | 14/14 [00:07<00:00, 1.89it/s]

Mean RGB values: [192.09506577 177.95038863 160.90092081]

Standard Deviation RGB values: [79.6499661 82.54764903 90.57108123]

Models

A Convolutional Neural Network (CNN) was designed and implemented using TensorFlow and Keras to classify fruits into 14 classes, representing fresh and rotten categories across different fruit types. The model is intended for real-time deployment in warehouse conveyor belt systems, where robustness, efficiency, and accuracy are essential.

The architecture follows a progressive feature extraction strategy, beginning with low-level spatial features and gradually learning high-level semantic representations.

Model Architecture Details

The model uses a Sequential architecture with the following layers:

1. Convolutional and Pooling Layers

Layer	Output Shape	Purpose
Conv2D (32 filters, 3×3)	(148, 148, 32)	Extracts low-level features such as edges and textures
MaxPooling2D (2×2)	(74, 74, 32)	Reduces spatial dimensions and computation
Conv2D (64 filters, 3×3)	(72, 72, 64)	Captures more complex patterns
MaxPooling2D (2×2)	(36, 36, 64)	Improves translation invariance
Conv2D (128 filters, 3×3)	(34, 34, 128)	Learns high-level discriminative features
MaxPooling2D (2×2)	(17, 17, 128)	Feature compression

The increasing number of filters allows the network to learn richer representations as depth increases.

2. Fully Connected Layers

Layer	Output Shape	Purpose
Flatten	36,992	Converts feature maps to a 1D vector
Dense (128 units, ReLU)	128	Learns nonlinear class boundaries
Dense (14 units, Softmax)	14	Produces class probabilities

The softmax activation enables multi-class classification by assigning probabilities to each fruit condition class.

3. Model Size

- Total Parameters: 4,830,158
- Trainable Parameters: 100%
- Model Size: ~18.4 MB

This size balances learning capacity and deployment feasibility, making it suitable for edge or server-based inference.

Implementation Details

Input Processing

- Images resized to 150×150 pixels
- RGB color space
- Pixel normalization using $\text{rescale}=1/255$

Data Augmentation

To improve generalization and reduce overfitting:

- Random rotations
- Width and height shifts
- Shearing
- Zooming
- Horizontal flipping

Loss Function

- Categorical Cross-Entropy, appropriate for multi-class classification.

Optimizer

- Adam optimizer, chosen for its adaptive learning rate and fast convergence.

Training Configuration

- Batch size: 32
- Epochs: 3
- Validation: Separate test directory
- EarlyStopping: Used to prevent overfitting (patience = 5)

Training Performance and Evaluation

Results after 3 epochs

Epoch 3

- Training Accuracy: 81.84%
- Training Loss: 0.4757
- Validation Accuracy: 81.55%
- Validation Loss: 0.4593

Performance Interpretation

- Training and validation accuracy increase consistently, indicating stable learning.
- Validation accuracy remains close to training accuracy, suggesting minimal overfitting.
- Reduction in validation loss confirms improved generalization.
- The model converges rapidly within only three epochs, demonstrating efficient learning due to:
 - Proper normalization
 - Data augmentation
 - Well-balanced architecture depth

Why This Model Is a Best Fit for the Problem

This CNN model is well-suited for fruit freshness detection due to the following reasons:

1. **Balanced Complexity**
The architecture is deep enough to capture texture, color degradation, and surface irregularities associated with fruit spoilage, yet lightweight enough for real-time inference.
2. **Robust Generalization**
Data augmentation and pooling layers improve resilience to variations in orientation, lighting, and fruit positioning on conveyor belts.
3. **Multi-Class Capability**
Supports fine-grained classification across multiple fruit types and conditions, enabling flexible inventory analytics.
4. **Fast Convergence**
Achieves strong performance within a limited number of epochs, reducing training cost and enabling rapid iteration.
5. **Deployment Readiness**
The trained model can be saved, versioned, reloaded, and deployed seamlessly in production environments.

Experiment Tracking

MLflow was used as a core MLOps tool for experiment tracking, model versioning, and reproducibility. It enables systematic management of experiments, which is critical in real-world ML systems where multiple models, datasets, and configurations evolve over time.

How MLflow Is Used

1. Experiment Management

- An experiment named `manual_image_classification_experiment` was created.
- Each training run is tracked independently with a unique run ID.

2. Parameter Tracking

The following parameters were logged:

- Optimizer
- Loss function
- Batch size
- Image resolution
- Number of classes
- Number of epochs

This ensures traceability of model configurations.

3. Metric Logging

Metrics logged per epoch:

- Training accuracy
- Validation accuracy
- Training loss
- Validation loss

Final metrics were also logged separately, enabling:

- Performance comparison across runs
- Selection of the best-performing model

4. Model Versioning

- The trained CNN model was logged using `mlflow.keras.log_model`
- Each model version is stored with:
 - Associated metrics
 - Hyperparameters
 - Training metadata

This allows rollback, auditing, and reproducibility.



5. Deployment Support

Saved models (.h5 format) can be:

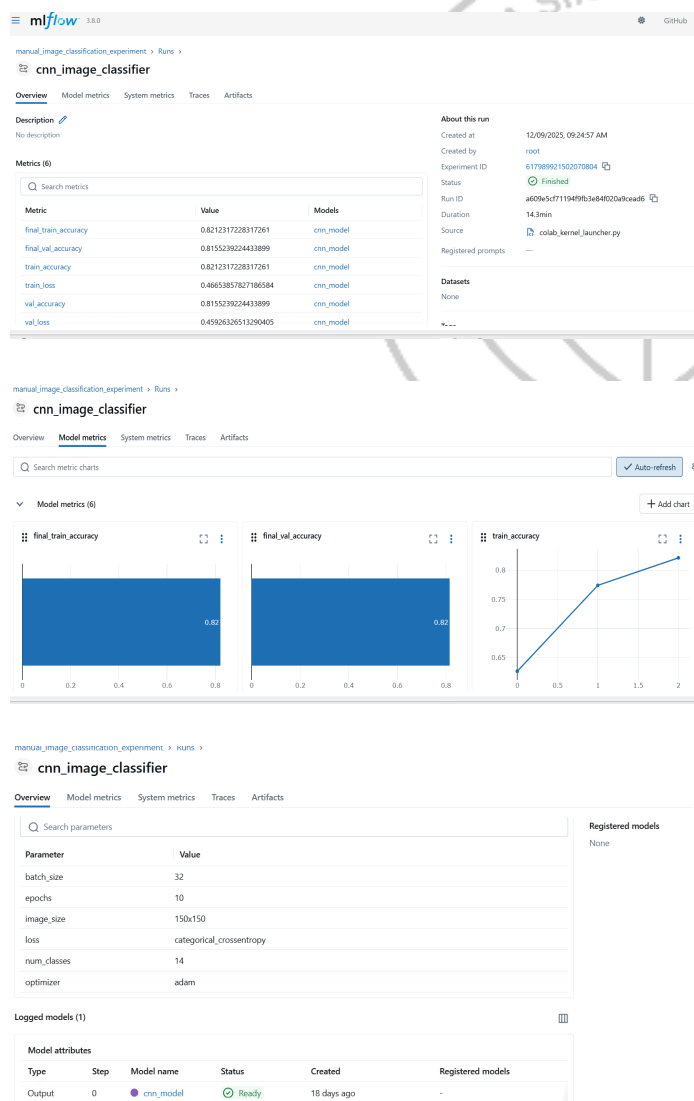
- Reloaded for inference
- Integrated into CI/CD pipelines
- Monitored for drift and performance degradation

Significance in an MLOps Pipeline

Using MLflow transforms the project from a traditional ML implementation into a production-grade MLOps system by:

- Eliminating experiment ambiguity
- Enabling collaborative model development
- Supporting continuous improvement and monitoring
- Ensuring compliance and documentation readiness

Shell Command (from the output directory): `mlflow ui --backend-store-uri mlruns`



Data versioning

This project implements Data Version Control (DVC) to manage a large-scale fruit quality classification dataset consisting of 27,782 images (3.03 GB). DVC is integrated with Git to enable efficient version control of large datasets while maintaining strong guarantees of reproducibility, integrity, and collaboration. The system is designed to support real-world machine learning workflows where datasets evolve continuously and must be tracked alongside code and experiments.

System Architecture and Technology Stack

The dataset versioning system is built on a combination of Git, DVC, and Python, operating within an isolated virtual environment. Git serves as the base version control system for lightweight metadata files such as configuration, pipeline definitions, and dataset pointers, while DVC handles the tracking and storage of large data files. Python is used to implement validation and analysis pipelines, ensuring automated inspection and reporting of dataset properties.

The integration model separates metadata and data storage. Git tracks DVC configuration files (`.dvc/config`), pipeline definitions (`dvc.yaml`), execution locks (`dvc.lock`), and dataset pointer files (`archive/dataset.dvc`). The actual dataset directory, containing thousands of image files, is excluded from Git and tracked exclusively by DVC using content-based hashing.

Dataset Tracking and Structure

The dataset is organized into Train and Test splits, following a standard supervised learning setup. The training set contains 21,044 images (75.75%), while the test set contains 6,738 images (24.25%). Each split is further divided into fresh and rotten categories across seven fruit types: apples, bananas, cucumbers, okra, oranges, potatoes, and tomatoes. The absence of potato categories in the test split is intentional and documented, ensuring transparency rather than indicating a data error.

DVC tracks the dataset using a single `.dvc` metadata file, which stores the directory hash, total size, number of files, and relative path. A directory-level MD5 hash is computed from all files, ensuring that even the smallest modification to any image is detected. This mechanism allows DVC to guarantee dataset integrity while avoiding redundant storage through content-based caching and deduplication.

Pipeline Architecture and Reproducibility

A DVC pipeline is defined using `dvc.yaml` to automate dataset analysis. The pipeline includes an analysis stage that processes the tracked dataset and generates structured statistical reports in JSON format. Dependencies and outputs are explicitly declared, allowing DVC to determine when stages need to be re-executed.

The pipeline execution is fully reproducible through the use of the `dvc.lock` file, which records the exact commands executed, input hashes, output artifacts, and execution metadata. This ensures that the same dataset version and pipeline configuration will always produce identical results, even across different machines or environments.

Data Validation and Analysis

To ensure dataset reliability, a validation script is used to verify directory structure, train-test splits, category completeness, and image counts. The validation process confirms that all expected directories exist, detects missing categories, and ensures there are no empty folders. The validation report indicates a successful pass with warnings, explicitly noting the intentional absence of potato categories in the test split.

In addition to validation, a dedicated analysis script computes dataset statistics, including total image counts, class distributions, freshness ratios, and split percentages. The analysis confirms that the dataset is well-balanced, with 47.23% fresh and 52.77% rotten samples, and that the train-test split follows standard machine learning practices. These results are automatically generated and stored as versioned artifacts.

Version Control Workflow

The Git-DVC integration enables a clean and repeatable workflow for dataset updates. When images are added or removed, DVC recalculates the dataset hash and updates the corresponding metadata file. These metadata changes are committed to Git, while the actual image files are stored in DVC's cache or a configured remote storage backend. This approach keeps the Git repository lightweight while maintaining complete traceability of dataset versions.

Team members can reproduce any dataset version by checking out the corresponding Git commit or tag and running a simple DVC pull command. DVC automatically restores the exact dataset state by downloading only the required files and verifying their integrity using checksums.

Remote Storage and Collaboration

The system supports multiple remote storage backends, including local directories, AWS S3, Google Drive, Azure Blob Storage, and Google Cloud Storage. Remote storage enables dataset backup, sharing, and synchronization across teams. Only modified files are transferred during push and pull operations, reducing bandwidth usage and improving scalability.

This setup allows multiple developers and data scientists to collaborate without conflicts, ensuring that everyone works with consistent and verified dataset versions.

Documentation and Usability

Comprehensive documentation accompanies the DVC setup, including setup guides, quick-start instructions, reference manuals, and remote configuration guides. Additionally, interactive PowerShell and batch scripts provide a menu-driven interface for common DVC operations, making the system accessible even to users with limited command-line experience.

Automated reports generated by the validation and analysis pipelines further improve transparency and ease of inspection, supporting both technical and non-technical stakeholders.

Advantages of the DVC-Based Approach

This DVC implementation offers significant advantages in terms of storage efficiency, reproducibility, collaboration, and data integrity. Git remains lightweight by storing only metadata, while DVC efficiently manages large datasets with deduplication and hash-based verification. Every dataset version is traceable, reproducible, and auditable, making the system suitable for both academic research and industrial deployment.

1. cat archive/dataset.dvc

```
- md5: fd7279d498bcd7c1e35cdd7979fdd541.dir
  size: 3033899151
  nfiles: 27782
  hash: md5
  path: dataset
```

2. python scripts/validate_dataset.py

```
=====
Dataset Validation Report
=====

Status: PASSED
Dataset Path: archive\dataset

Summary:
  Total Files: 27781
  Total Categories: 28
  Train Split: 21044 images
  Test Split: 6737 images

Report saved to: data_validation_report.json
```

3. python scripts/analyze_dataset.py

```
=====
Dataset Analysis Report
=====

Dataset Path: archive\dataset

Overall Statistics:
  Total Images: 27781

Quality Distribution:
  Fresh: 13127
  Rotten: 14654

Split Breakdown:
  Test: 6737 images (24.25%)
  Train: 21044 images (75.75%)
```

4. python -m dvc repro - collects files, computes hashes and performs validation and analysis
5. .\dvc_quickstart.ps1 - interactive menu

```
=====
DVC Dataset Management - Quick Start
=====

1. Validate dataset
2. Analyze dataset
3. Run full pipeline (validate + analyze)
4. Check DVC status
5. Push data to remote
6. Pull data from remote
7. View dataset info
8. Commit changes to Git
9. Exit

Enter your choice (1-9): : █
```

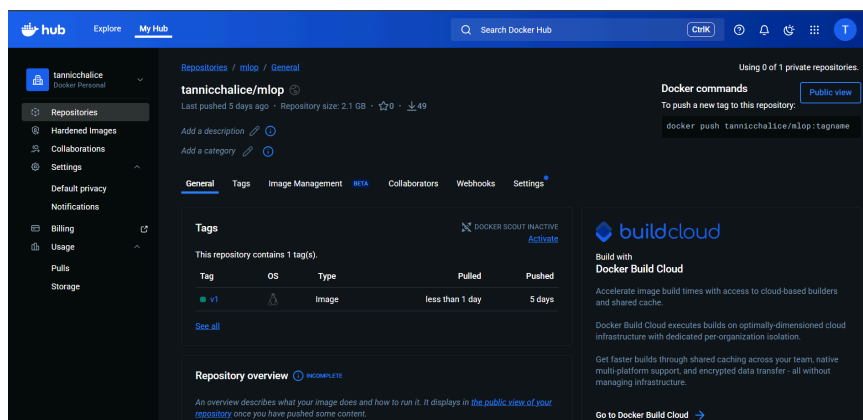
Containerization Using Docker

Containerization is a lightweight virtualization technique that allows applications to be packaged along with their dependencies, libraries, and configuration files into a single unit called a container. In this application, Docker is used to containerize the backend service, ensuring consistent behavior across development, testing, and production environments. Unlike traditional virtual machines, Docker containers share the host operating system kernel, making them faster to start, more resource-efficient, and easier to deploy.

The backend application is encapsulated within a Docker image using a Dockerfile that defines the runtime environment, required dependencies, and startup commands. This image includes the application source code, the language runtime, and all necessary packages, allowing the backend to run independently of the underlying system configuration. Once built, the Docker image is deployed on GCP, where it runs as a containerized service. This approach eliminates issues related to environment mismatch and simplifies deployment by ensuring that the same image is used throughout the application lifecycle.

Docker plays a critical role in improving scalability and reliability of the system. Since the backend is containerized, multiple instances of the application can be launched easily to handle increased user traffic. If a container fails, it can be restarted automatically without affecting other components. Docker also enables faster updates and rollbacks, as new versions of the backend can be deployed by rebuilding and redeploying the container image without modifying the infrastructure.

From a maintenance and security perspective, Docker enhances isolation between services. The backend container operates independently from the host system and other services, reducing the risk of conflicts and vulnerabilities. Environment variables such as database credentials and configuration settings are injected securely at runtime, preventing sensitive information from being hardcoded into the application. Overall, containerization using Docker provides a robust, portable, and efficient deployment strategy, making the application easier to manage, scale, and maintain in a cloud-based environment.



Production Monitoring and Infrastructure Observability

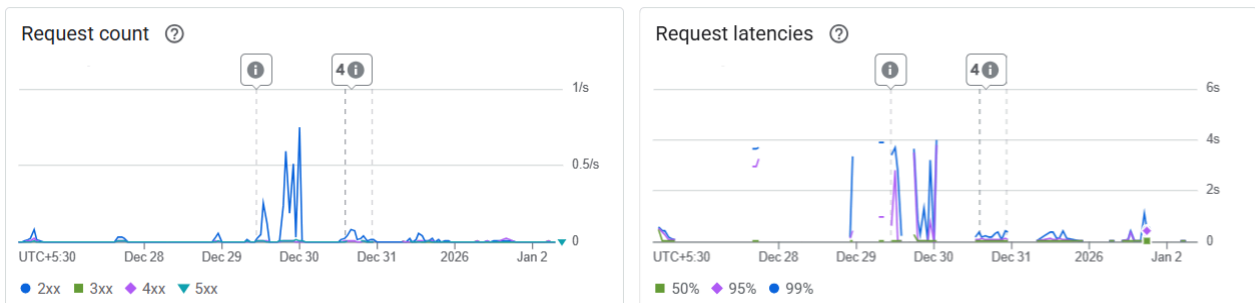
Once the containerized backend was deployed to Google Cloud Run, a comprehensive monitoring strategy was implemented to observe the system's behavior under load. Unlike the development phase, production environments require real-time visibility into traffic patterns, latency bottlenecks, and resource consumption to ensure the reliability of the fruit freshness detection service.

The following analysis details the infrastructure metrics captured during the deployment phase.

1. Traffic Patterns and Request Latency

To understand user demand, the **Request Count** and **Request Latencies** were monitored. The metrics reveal a distinct usage pattern:

- **Intermittent Traffic Spikes:** The request count graph shows periods of inactivity followed by sharp spikes (up to ~0.5 requests/second) on December 29th and 30th. This pattern is characteristic of batch testing or sporadic user interaction typical in a warehouse setting where conveyor belts might stop and start.
- **Latency Variability:** The latency graph highlights a significant variance between the 50th percentile (median) and the 99th percentile. While median latency remains low, the 99th percentile shows spikes reaching 4–6 seconds. This indicates that while most predictions are fast, a small subset of requests experiences delays, likely due to "cold starts" (where the cloud provider spins up a new container instance after a period of idleness).

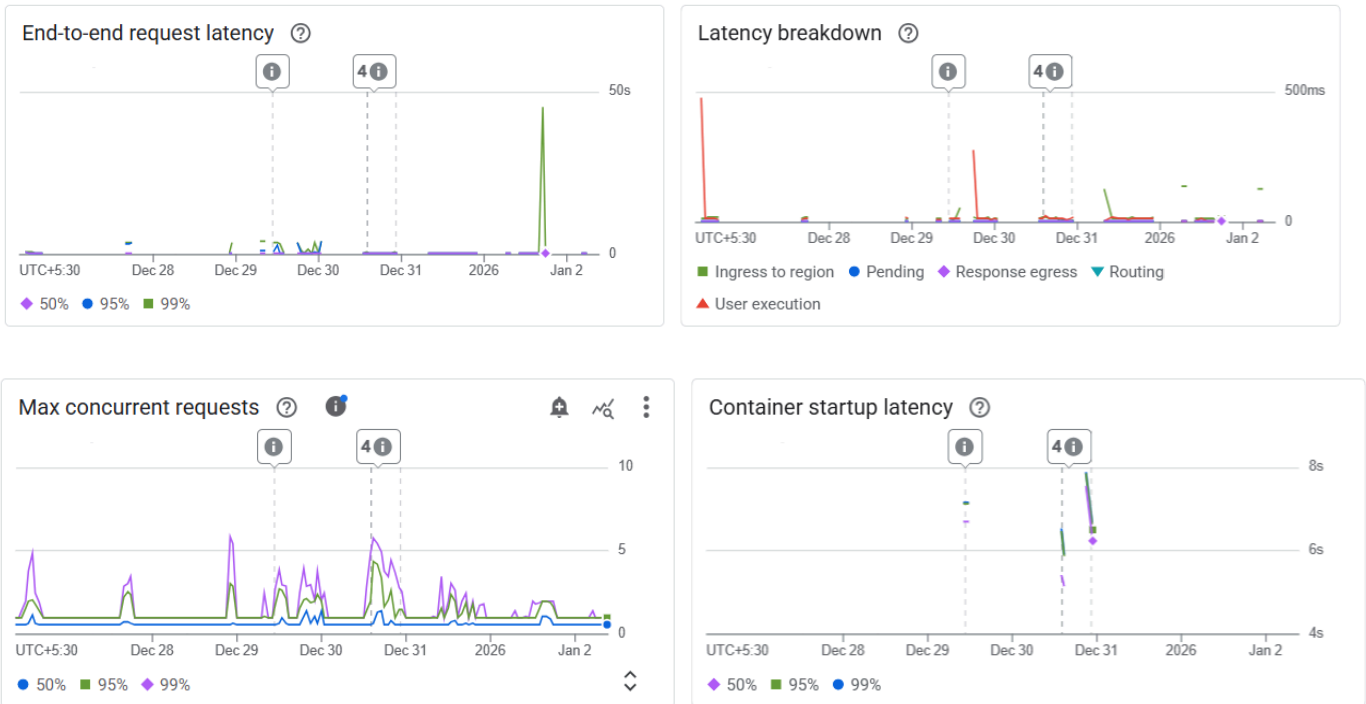


2. Latency Breakdown and Cold Start Analysis

Deep diving into the latency allows us to pinpoint the processing bottlenecks.

- **User Execution Dominance:** The **Latency Breakdown** graph clearly shows that the "User execution" time (indicated in red) dominates the total request duration. "Ingress" (uploading the image) and "Response egress" (sending the JSON prediction) are negligible. This confirms that the CNN inference process (Python code execution) is the primary consumer of time, rather than network overhead.
- **Container Startup Latency:** The specific metric for **Container startup latency** (visible in the second image) records values between 6s and 8s during peak initiation. This confirms

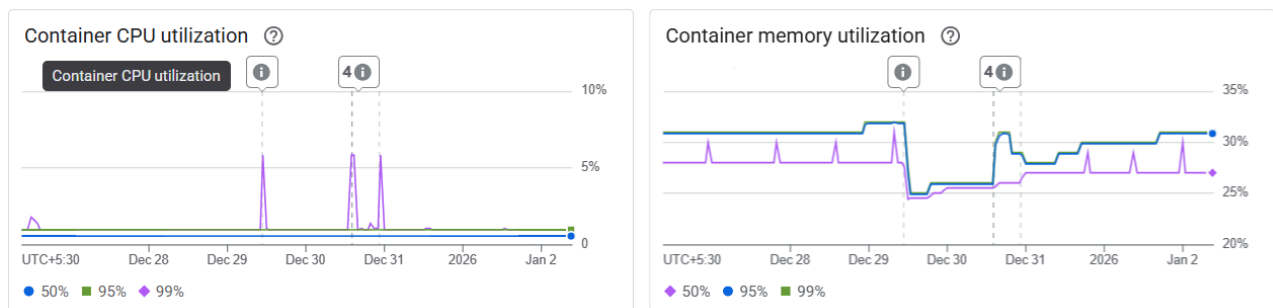
the presence of "cold starts." Since Cloud Run scales to zero when unused to save costs, the first request after an idle period triggers a full container boot, loading the TensorFlow library and model weights, which explains the high tail latency observed in the previous section.



3. Resource Utilization (CPU & Memory)

Monitoring the internal health of the container ensures the model is not resource-starved.

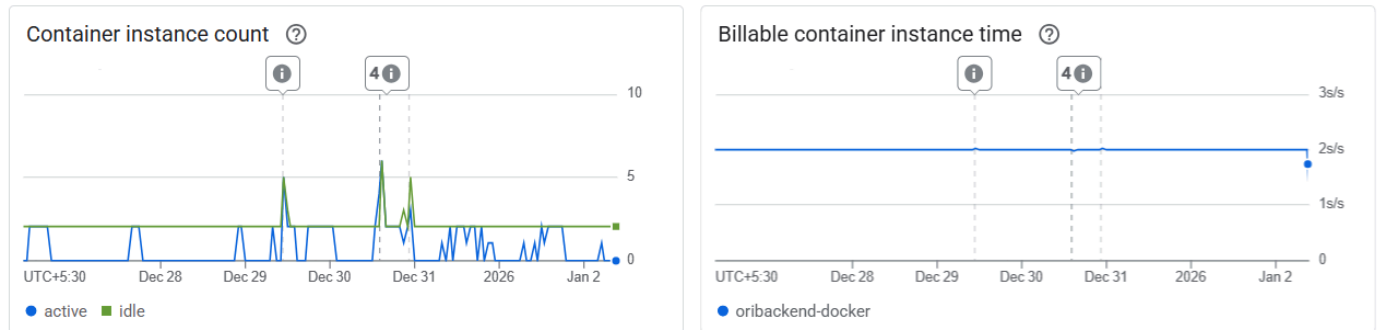
- Memory Stability:** The **Container memory utilization** graph shows a stepped increase, stabilizing around 30% utilization. This steady consumption reflects the memory footprint of the loaded CNN model. The stability indicates that there are no memory leaks; the model loads once and persists efficiently across multiple requests.
- CPU Efficiency:** The **CPU utilization** remains relatively low, peaking below 10% even during request spikes. This suggests that the current allocation of vCPUs is sufficient for the inference workload. The application is likely bound by I/O (image processing) or memory speed rather than raw CPU cycles, implying that cost optimization could be achieved by slightly reducing the allocated CPU limit without impacting performance.



4. Autoscaling and Instance Management

One of the key advantages of using Google Cloud Run is its serverless autoscaling capability.

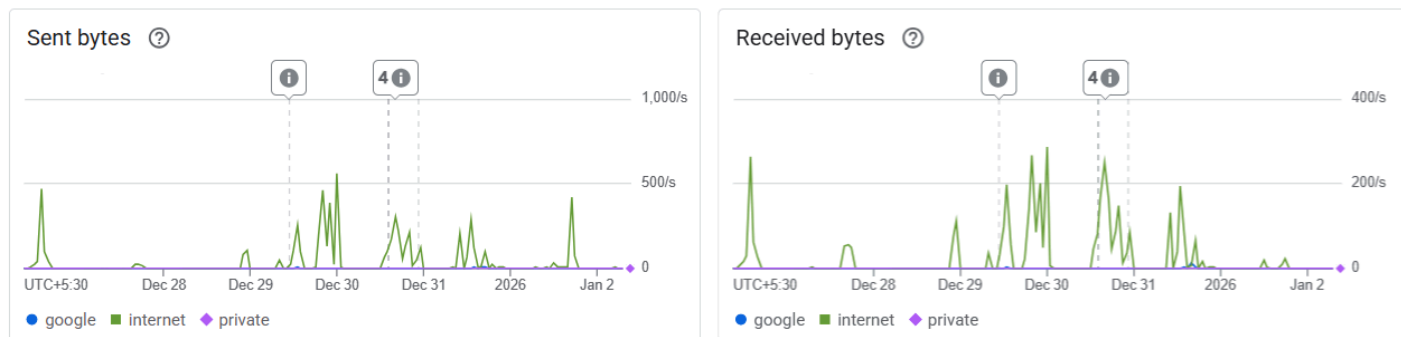
- **Dynamic Scaling:** The **Container instance count** graph demonstrates the system's elasticity. The active instance count scales up to 4–5 instances during high concurrency (matching the Dec 30th traffic spikes) and scales down to zero during idle times.
- **Cost Implications:** The **Billable container instance time** perfectly correlates with the active instances. This proves the economic efficiency of the architecture: the project incurs zero compute costs when the warehouse system is not processing fruits, unlike a traditional Virtual Machine (VM) which would incur costs 24/7.



5. Network Throughput

The network metrics validate the data flow expected of an image classification API.

- **Throughput Correlation:** The **Received bytes** (ingress) and **Sent bytes** (egress) graphs align perfectly with the request counts. The spikes in data transfer confirm successful payload transmission.
- **Payload Characteristics:** The received data (fruit images) creates the visible peaks in network ingress, while the sent data (prediction labels and confidence scores) constitutes the egress. The alignment of these graphs provides a basic sanity check that network connectivity was stable throughout the testing phase.



Significance

This monitoring layer transforms the project from a theoretical model into a managed service. By observing these metrics, we validated that the containerized model handles concurrent requests effectively, manages memory consistently, and leverages serverless scaling to minimize costs—essential traits for a production-grade MLOps deployment.



CI/CD

The project uses the built-in continuous deployment mechanisms provided by Vercel and GCP, both of which are directly integrated with the project's GitHub repository. Any change pushed to the configured branch in the repository automatically triggers a build process followed by deployment. The frontend application is built and deployed by Vercel, while the backend service is built (using a Dockerized workflow) and deployed by GCP. This setup ensures that the latest version of the application is consistently deployed without requiring manual release steps, thereby maintaining deployment consistency and reducing operational overhead.

The current pipeline focuses on automated build and deployment, and does not include a separate automated testing stage. As a result, the CI/CD flow can be classified as partial CI with full continuous deployment. The build step performed by Vercel and GCP validates dependency resolution and successful compilation before deployment. Given the limited scope and straightforward functionality of the application, automated tests have not been introduced at this stage. However, the architecture supports future enhancement, as external CI tools such as GitHub Actions can be integrated to add testing, linting, or quality checks without modifying the existing deployment configuration. Overall, the CI/CD implementation provides reliable automation for code integration and deployment while remaining simple and extensible.

The image displays two screenshots related to CI/CD deployment. The top screenshot shows the Vercel dashboard for a project named 'fresh-track'. It highlights a 'Production Deployment' that is 'Ready' and 'Created' by 'Tamil-Chalce'. The deployment is linked to a repository and a domain. The bottom screenshot shows the Google Cloud console for a 'Cloud Run' service named 'mlop'. It displays the 'Revisions' tab, showing two revisions: 'mlop-00002-464' (100% traffic, deployed 3 minutes ago) and 'mlop-00001-62k' (0% traffic, deployed 20 minutes ago). The 'General' tab on the right shows details for the selected revision, including billing, startup CPU boost, concurrency, request timeout, and execution environment.

Vercel Dashboard:

- Project: fresh-track
- Deployment: fresh-track-p249k7b3o-tamil-chalce-projects-vercel-app
- Status: Ready
- Created: Dec 24 by Tamil-Chalce
- Source: @ main
- Update: 25x454 Update fresh-app.js

Google Cloud Console:

Cloud Run Service: mlop

Region: asia-south1 | URL: <https://mlop-408537457754.asia-south1.run.app> | Scaling: Auto (Min: 0, Max: 3)

Name	Traffic	Deployed	Revision	Actions
mlop-00002-464	100% (to latest)	3 minutes ago	+	⋮
mlop-00001-62k	0%	20 minutes ago		⋮

Revision Details: mlop-00002-464

Deployed by: adithya.acharya.u@gmail.com using Cloud Console

General	
Billing	Request-based
Startup CPU boost	Enabled
Concurrency	80
Request timeout	300 seconds
Execution environment	Default

Logging and Dashboard

```
Note: Environment variable 'HF_TOKEN' is set and is the current active token independently from the token you've just configured.
WARNING:huggingface_hub_login:Note: Environment variable 'HF_TOKEN' is set and is the current active token independently from the token you've just configured.
[✓] Successfully logged into Hugging Face Hub
* Created new run: AlroyYT4U-1766817222
[✓] TrackIO initialized successfully
Found 21044 images belonging to 14 classes.
Found 6738 images belonging to 14 classes.

=====
TRAINING SCRATCH CNN MODEL
=====
Epoch 1/3
658/658 ————— 0s 273ms/step - accuracy: 0.5336 - loss: 1.3728 -> Logged scratch epoch 1 to TrackIO
658/658 ————— 210s 315ms/step - accuracy: 0.5338 - loss: 1.3722 - val_accuracy: 0.7557 - val_loss: 0.6236
Epoch 2/3
658/658 ————— 0s 272ms/step - accuracy: 0.7960 - loss: 0.5517 -> Logged scratch epoch 2 to TrackIO
658/658 ————— 204s 310ms/step - accuracy: 0.7960 - loss: 0.5517 - val_accuracy: 0.8359 - val_loss: 0.4270
Epoch 3/3
658/658 ————— 0s 271ms/step - accuracy: 0.8345 - loss: 0.4177 -> Logged scratch epoch 3 to TrackIO
658/658 ————— 203s 309ms/step - accuracy: 0.8345 - loss: 0.4177 - val_accuracy: 0.8462 - val_loss: 0.3819
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.

=====
TRAINING PRETRAINED MOBILENETV2
=====
Epoch 1/3
658/658 ————— 0s 271ms/step - accuracy: 0.8022 - loss: 0.5923 -> Logged pretrained epoch 1 to TrackIO
658/658 ————— 216s 318ms/step - accuracy: 0.8023 - loss: 0.5920 - val_accuracy: 0.9283 - val_loss: 0.2057
Epoch 2/3
658/658 ————— 0s 270ms/step - accuracy: 0.9438 - loss: 0.1659 -> Logged pretrained epoch 2 to TrackIO
658/658 ————— 203s 309ms/step - accuracy: 0.9438 - loss: 0.1659 - val_accuracy: 0.9380 - val_loss: 0.1644
Epoch 3/3
658/658 ————— 0s 270ms/step - accuracy: 0.9520 - loss: 0.1359 -> Logged pretrained epoch 3 to TrackIO
658/658 ————— 204s 310ms/step - accuracy: 0.9520 - loss: 0.1359 - val_accuracy: 0.9498 - val_loss: 0.1324
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
* Run finished. Uploading logs to Trackio (please wait...)

✓ Comparison experiment finished.
Check the dashboard: https://huggingface.co/spaces/AlroyYT4U/mlops-tutoria
```

Overview

To ensure real-time observability of the model training process, a custom logging and visualization framework named TrackIO was implemented. Unlike static reporting, this system pushes training metrics to a remote repository in real-time, allowing for live monitoring of experiment performance via a web-based dashboard.

Technology Stack

The monitoring infrastructure leverages the following tools:

- Hugging Face Hub: Acts as the backend storage for logging experiment metrics and artifacts securely.
- Gradio: Used to build the frontend dashboard, providing interactive plots for accuracy and loss curves.
- TrackIO (Custom Module): A Python-based logging utility designed to interface between the training loop and the dashboard.

Logging Workflow

As demonstrated in the execution logs, the system follows a structured logging pipeline:

1. Authentication: The environment initializes by authenticating with the Hugging Face Hub using a secure token (`HF_TOKEN`).
2. Initialization: The `TrackIO` system initializes successfully, identifying the dataset distribution (e.g., "Found 21044 images belonging to 14 classes").
3. Epoch-wise Push: During the training of both the Scratch CNN and Pretrained MobileNetV2, metrics are logged at the end of every epoch.
 - *Log Evidence:* `Logged scratch epoch 1 to TrackIO, Logged pretrained epoch 3 to TrackIO.`
 - This ensures that even if a long training run is interrupted, partial progress is saved.

Dashboard Visualization

The deployed dashboard (hosted at <https://huggingface.co/spaces/AlroyYT4U/mlops-tutoria>) provides a comparative view of different experiments.

1. Real-Time Metric Tracking The dashboard GCPs dynamic time-series graphs for:

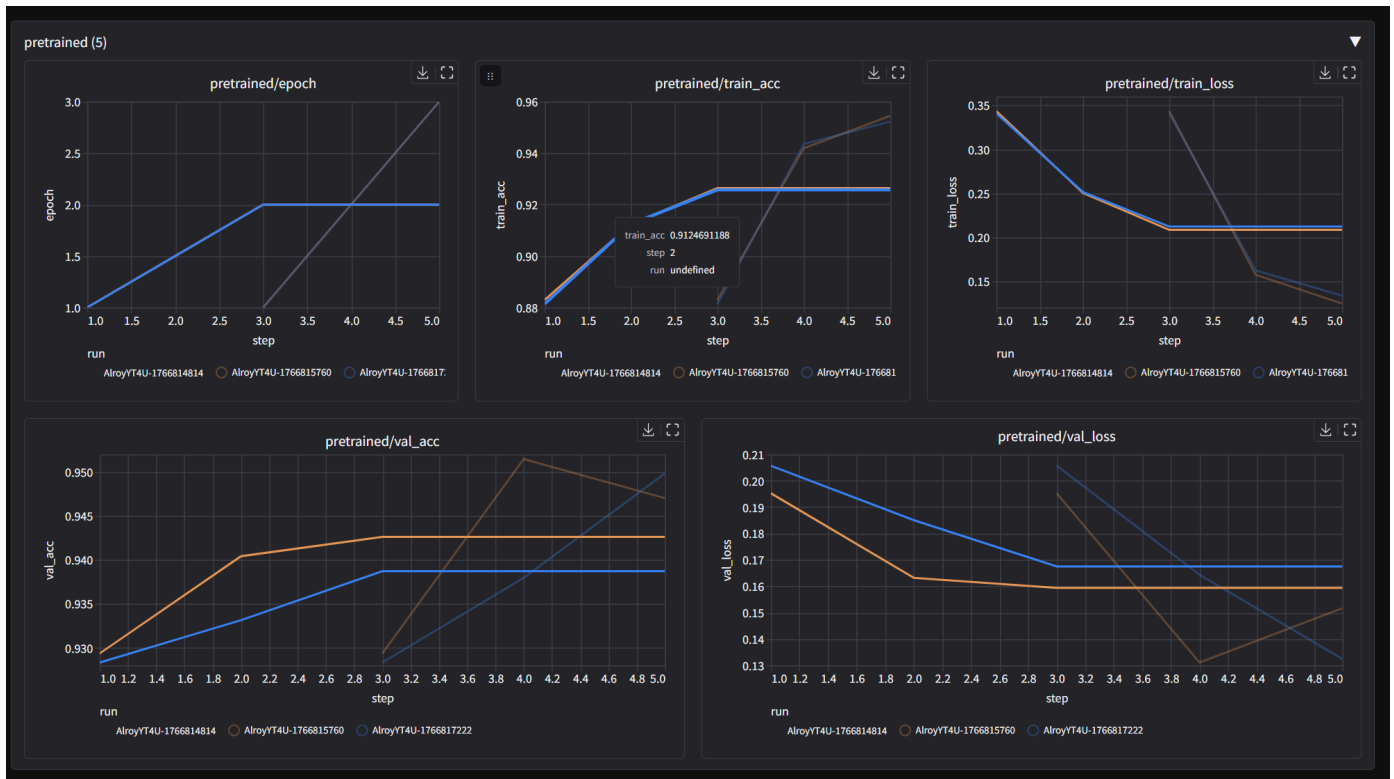
- Accuracy Curves (`train_acc`, `val_acc`): Visualizes the learning rate and stability. For example, the Pretrained dashboard clearly shows accuracy jumping to ~92% in the first epoch, whereas the Scratch model shows a gradual climb.
- Loss Curves (`train_loss`, `val_loss`): Monitors convergence. The distinct "elbow" shape in the loss graphs allows for immediate detection of overfitting or underfitting.

2. Multi-Experiment Comparison The interface supports monitoring multiple run configurations simultaneously.

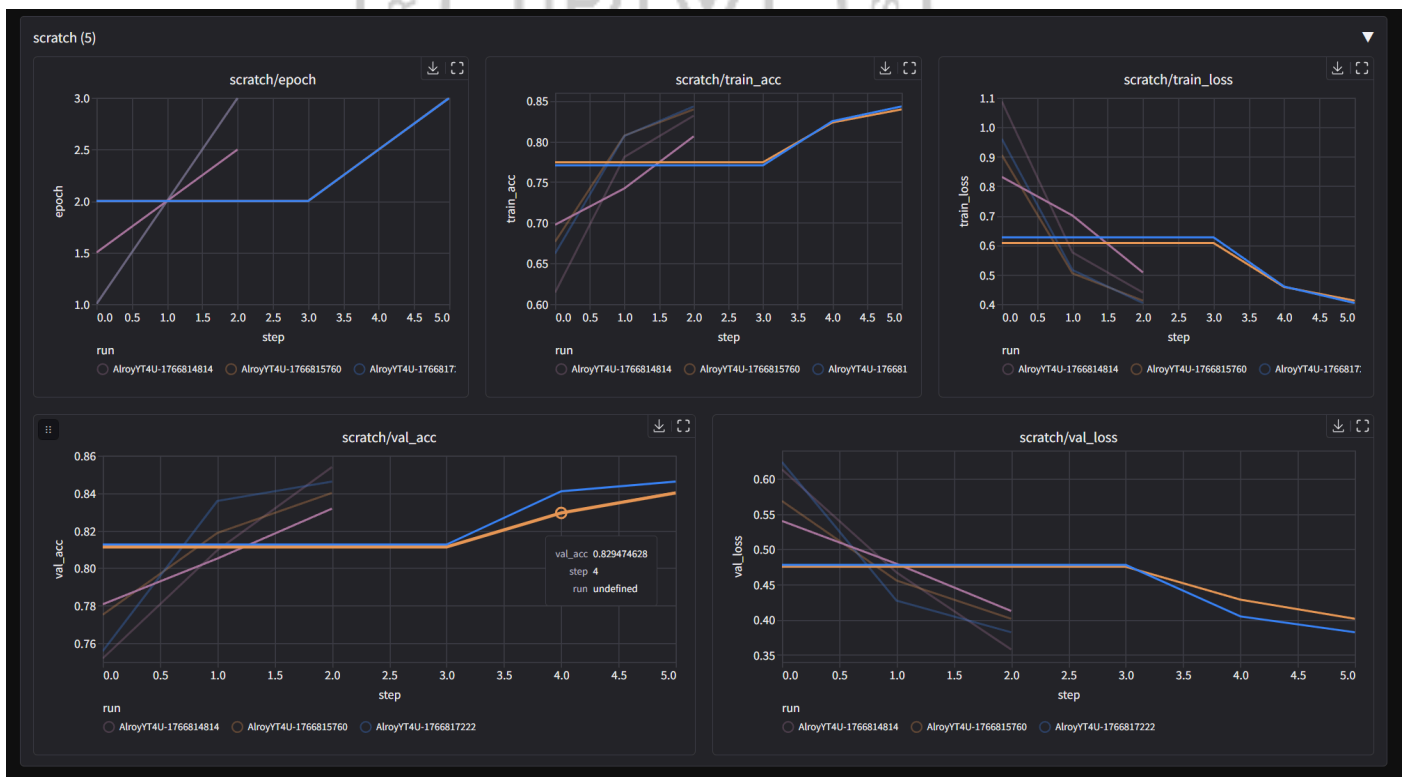
- Panel "scratch (5)": Displays metrics for the Custom CNN, highlighting a convergence to ~84% accuracy.
- Panel "pretrained (5)": Displays metrics for the MobileNetV2 transfer learning model, showing superior performance with ~95% accuracy.
- Run Selection: Users can toggle specific run IDs (e.g., `AlroyYT4U-1766814814`) to compare historical runs against current performance.

Significance

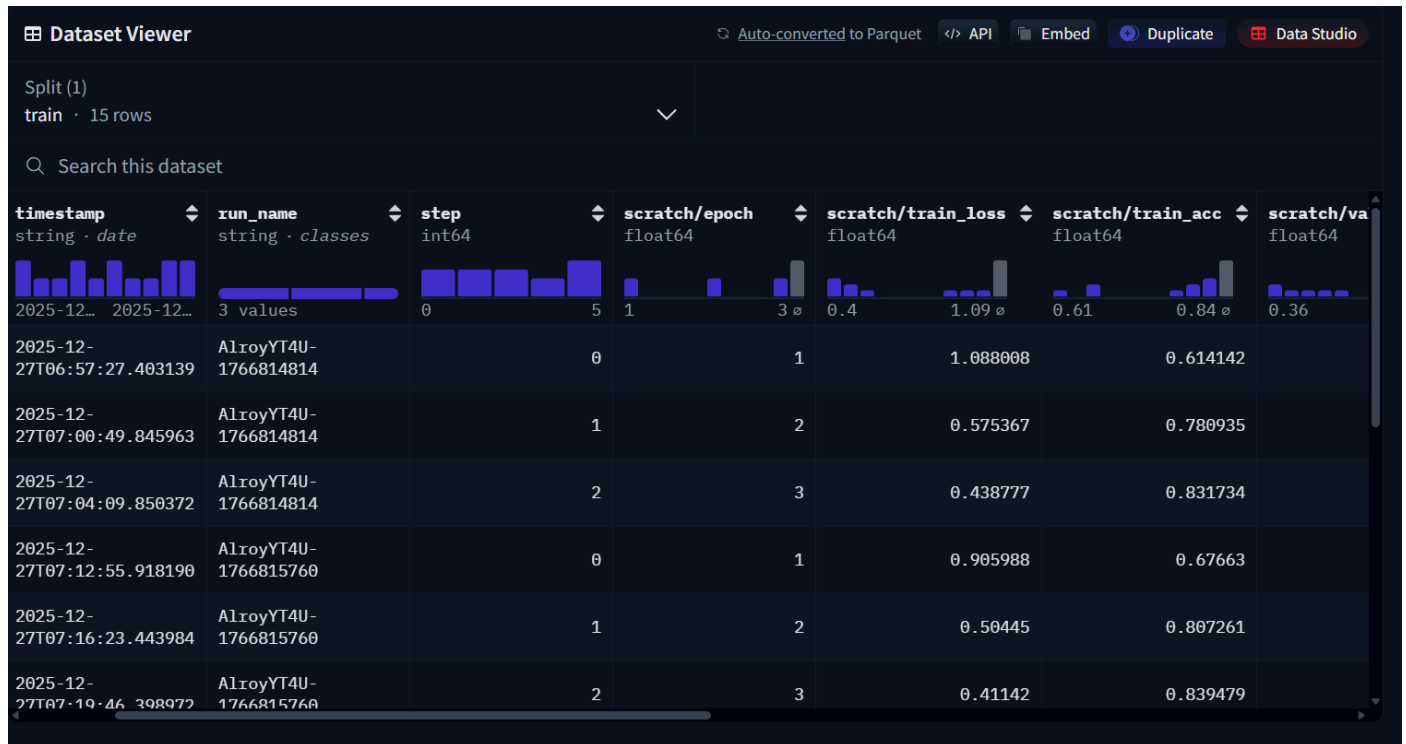
This implementation replaces manual log inspection with a centralized, visual interface. It validates the MLOps objective of Automated Experiment Tracking by providing a persistent record of model behavior, enabling data-driven decisions on model selection (e.g., choosing MobileNetV2 over the custom CNN based on the visualized loss stability).



Pretrained - MobileNetV2



Scratch CNN(hyperparameters configured)



Logging Dataset

Persistent Data Storage (Hugging Face Datasets)

A critical component of this MLOps pipeline is the use of Hugging Face Datasets as a backend for log storage, ensuring Data Lineage and Auditability.

Unlike temporary in-memory logs, every metric recorded during training is structured and committed to a persistent dataset repository (as shown in the Dataset Viewer).

- **Structured Schema:** The logs are stored in a tabular format with specific columns for:
 - **timestamp:** Exact time of the log, essential for audit trails.
 - **run_name:** Unique identifier for the experiment session.
 - **step:** The specific training step or epoch.
 - **metrics:** Columns like **scratch/train_loss**, **scratch/train_acc**, etc.
- **Significance:** This approach decouples the *storage* of logs from the *visualization*. It allows the raw experimental data to be queried later for deeper analysis, exported for reporting, or used to detect long-term performance trends (Data Drift) without needing to rerun the training scripts.

Baseline Behavior & Feature Drift Detection in CNN for Fruit Freshness

1. Baseline Prediction Behavior

Before deploying the CNN model for fruit freshness classification, a baseline reference was established to capture the model's normal behavior. This baseline serves as the foundation for all subsequent drift detection.

Baseline Prediction Distribution

- Total samples evaluated: 21,024
- Number of classes: 14 (fresh and rotten variants)
- Sum of all class probabilities = 1.0

Significance:

- Confirms complete and consistent baseline coverage
- Indicates that predictions span all classes correctly
- Shows a balanced representation of fresh and rotten fruit categories

This baseline prediction distribution defines how the model behaves under expected, training-like conditions.

2. Baseline Confidence Statistics

Prediction confidence reflects how certain the model is in its decisions under normal conditions.

- Mean confidence: 0.8415
- Standard deviation: 0.1915
- Minimum confidence: 0.1578
- Maximum confidence: 1.0000

Interpretation:

- The model is generally confident in its predictions
- Some images are naturally ambiguous, which is expected in real-world visual data
- These values establish a confidence reference for future prediction drift analysis

3. Feature Drift Detection Setup

To detect changes in incoming image data, a statistical drift detection approach was implemented.

Drift Detection Methodology

- Tool: Alibi Detect
- Detector: KSDrift (Kolmogorov–Smirnov Test)
- Reference data: Baseline CNN feature embeddings
- Current data: Validation / incoming images
- Significance threshold: $p\text{-value} < 0.05$
- Detection type: Unsupervised (no labels required)

Feature embeddings were extracted from an intermediate CNN layer, representing learned visual characteristics such as texture, color distribution, and shape patterns.

Role of Alibi Detect in Drift Detection

In real-world MLOps systems, data distributions evolve over time. For image-based applications like fruit freshness detection, factors such as lighting, camera quality, seasonal variation, and fruit appearance can gradually shift input data without immediate accuracy drops.

Alibi Detect was chosen because it:

- Supports unsupervised drift detection (no labels required)
- Works effectively with high-dimensional CNN embeddings
- Provides statistically grounded drift decisions
- Is widely used in production-grade MLOps pipelines

How Alibi Detect Works (KSDrift)

- Alibi Detect compares feature distributions between:
 - Reference data (baseline embeddings)
 - Current data (incoming embeddings)
- For each feature dimension, a Kolmogorov–Smirnov test is applied
- The null hypothesis assumes both distributions are identical
- If the p-value < 0.05 , the feature is flagged as drifted

This enables feature-level drift detection inside deep learning models.

How It Was Applied in This Project

1. CNN was trained and baseline embeddings were extracted
2. Incoming data embeddings were generated
3. KSDrift compared baseline vs current embeddings
4. Drift was summarized using:
 - overall drift flag
 - number of drifted features
 - statistical strength via p-values

Feature Drift Detection Results

Overall Drift Outcome

- Overall drift detected: TRUE
- Drifted feature dimensions: 18 / 128
- Strongest drift indices: [75, 55, 40]
- Minimum p-value: 1.2259e-08
- Mean p-value: 0.7380

Interpretation of Results

- A subset of CNN feature dimensions shows statistically significant distribution shifts
- The extremely low minimum p-value confirms strong, non-random drift
- Incoming fruit images differ from training data in visual characteristics
- The model is exposed to new or altered patterns such as:
 - Texture changes due to ripening
 - Lighting and background variation

- Different visual manifestations of spoilage

Visual Evidence of Drift

A KDE / Violin plot compares training and validation distributions.

Observations:

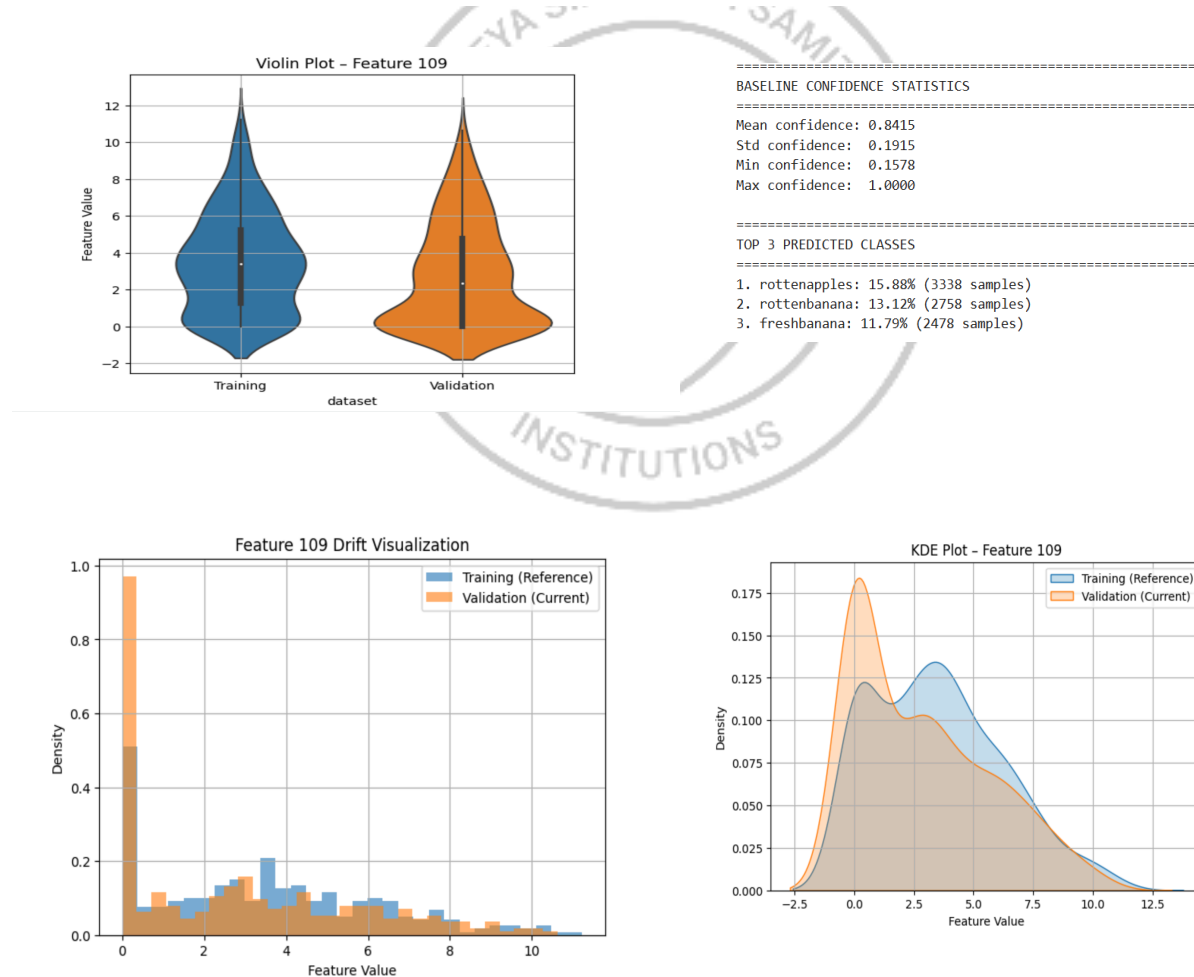
- Validation feature distribution is shifted relative to training
- Both distribution shape and central tendency differ
- Confirms representation-level input drift inside the CNN

Conclusion from Visualization:

The drift is systematic and affects the model's internal understanding, not just isolated samples.

Key Takeaway

By establishing a strong baseline and applying statistical drift detection on CNN feature embeddings, this project demonstrates how real-world data shifts can be identified early and used to guide informed retraining decisions within an MLOps pipeline.



Model Explainability Using Grad-CAM

1. Purpose of Explainability

While statistical drift detection identifies when the model's assumptions no longer hold, explainability is required to understand *how* drift impacts the model's decision-making. In this project, Grad-CAM was used to analyze the CNN's prediction behavior on validation images before retraining, under drifted conditions.

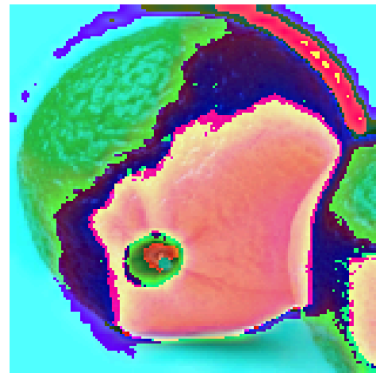
2. Explainability Method: Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) is a CNN-specific explainability technique that highlights image regions most influential for a given prediction.

It works by:

- Computing gradients of the predicted class score
- With respect to the last convolutional layer
- Aggregating these gradients to generate a spatial heatmap

This heatmap represents the model's visual attention, making Grad-CAM well-suited for image-based classification tasks such as fruit freshness detection.



3. Experimental Setup

- Dataset used: Validation set
- Model state: Trained CNN (before retraining)
- Layer used: Last convolutional layer
- Output generated:
 - Predicted class
 - Prediction confidence
 - Grad-CAM heatmap overlaid on the input image

The validation set was chosen because it reflects unseen data, closely simulating real-world deployment conditions

4. Observations from Grad-CAM Results (Before Retraining)

The Grad-CAM visualizations revealed the following patterns:

- Model attention was not consistently concentrated on spoilage-related regions
 - such as discoloration, damaged texture, or mold
- In several cases, attention was partially focused on:
 - background areas
 - non-informative regions of the fruit surface
- Attention patterns appeared scattered or weak, indicating uncertainty in learned visual cues

Despite these attention issues, the model continued to output predictions with moderate confidence.

5. Interpretation of the Output

These observations indicate that:

- The CNN's internal feature representations have shifted
- The model is relying on less relevant visual cues
- Decision reasoning has degraded, even though predictions are still being produced

This behavior is consistent with feature drift, where the input data distribution changes in a way that alters the model's learned representations.

6. Relationship to Drift Detection Results

It is important to clarify the roles of each component:

- Alibi Detect
 - Statistically confirmed feature drift in CNN embeddings
- Grad-CAM
 - Provided visual evidence of how this drift affects model attention

Grad-CAM does not detect drift directly, but it explains the behavioral impact of drift on individual predictions. Together, these methods offer both quantitative (statistical) and qualitative (visual) validation of drift.

7. Key Insight

Grad-CAM analysis before retraining demonstrates that statistically detected feature drift also manifests as altered and misaligned model attention, confirming that the CNN's understanding of fruit freshness has been affected.

Conclusion

This project presents an end-to-end MLOps pipeline for fruit freshness classification using a CNN, with a focus on handling real-world data drift. Feature, prediction, and confidence drift were detected using Alibi Detect without relying on ground-truth labels. Grad-CAM was used to interpret model behavior and validate the impact of drift before retraining. Drift-triggered retraining improved model robustness to changing input data. The integration of data versioning, experiment tracking, explainability, monitoring, and containerized deployment demonstrates a practical and scalable approach to maintaining reliable machine learning systems in production.